

SC4055 Introduction to Quantum Computing
NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-28 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

1	The Qubit: Beyond the Bit	1
1.1	Intuition: Why a Bit Is Not Enough	1
1.2	Formalism: The Qubit	1
1.3	Measurement and Collapse	2
1.4	Multi-Qubit Preview	2
1.5	Python: Qubits with NumPy	3
1.6	Density Matrices and Global Phase	4
1.7	Exercises	4
2	Quantum Circuits: Gates and Entanglement	5
2.1	Intuition: Reversible Rotations	5
2.2	Single-Qubit Gates	5
2.3	Two-Qubit Gates and Entanglement	6
2.4	Universality	6
2.5	Python: Gates and Bell States	7
2.6	Measurement, No-Cloning and Teleportation Preview	8
2.7	Exercises	8
3	Early Quantum Advantage: Deutsch–Jozsa and Bernstein–Vazirani	9
3.1	Intuition: The Power of One Query	9
3.2	Phase Kickback and the Oracle	9
3.3	Deutsch and Deutsch–Jozsa	10
3.4	Bernstein–Vazirani	10
3.5	Python: Simulating DJ and BV	11

3.6	Why Oracular Separations Matter	12
3.7	Simon and Period Structure	12
3.8	Implementation Cost and Practical Notes	12
3.9	Exercises	13
4	Grover's Search: Amplitude Amplification	14
4.1	Intuition: Rotating Toward the Needle	14
4.2	The Oracle and Diffusion	14
4.3	Optimal Number of Iterations	15
4.4	Amplitude Amplification	15
4.5	Geometric View and Fixed-Point Variants	16
4.6	Applications Beyond Search	16
4.7	Error Analysis and Reality	16
4.8	Beyond Unstructured Search: Structured Variants	16
4.9	Variations and History	16
4.10	Python: Grover Simulation	17
4.11	Query Lower Bounds	17
4.12	Exercises	18
5	Shor's Algorithm: Period Finding and Factoring	19
5.1	Intuition: Factoring = Period Finding	19
5.2	Quantum Fourier Transform	19
5.3	Shor's Period-Finding Circuit	20
5.4	Example: Factoring 15	20
5.5	Continued Fractions and Classical Post-Processing	20
5.6	Shor vs. Grover: Exponential vs. Quadratic	21
5.7	Resource Estimates	21
5.8	Historical Impact and Cryptography	21
5.9	Python: QFT and Period Finding	21
5.10	From Period to Factor: Number Theory Step	22
5.11	Exercises	23

6	Noise, Decoherence and Quantum Error Correction	24
6.1	Intuition: Why Quantum Is Fragile	24
6.2	Decoherence and Noise Models	24
6.3	Three-Qubit Codes	25
6.4	Surface Code and Threshold	25
6.5	Python: Noise and Repetition Code	25
6.6	Stabilizer Formalism in Brief	27
6.7	Exercises	27
7	Quantum Complexity and Supremacy	28
7.1	Intuition: The Complexity Zoo	28
7.2	BQP and Friends	28
7.3	Quantum Supremacy / Advantage	29
7.4	Oracular Separations	30
7.5	Beyond BQP: QMA and Local Hamiltonian	30
7.6	Dequantization and Quantum Inspirations	30
7.7	Interactive Proofs and Delegation	30
7.8	Fine-Grained and Average-Case Hardness	30
7.9	Open Problems	31
7.10	Proof Techniques	31
7.11	Python: Complexity Scaling Demo	31
7.12	Unconditional vs. Conditional Gaps	32
7.13	Exercises	32
8	Applications: QML, Chemistry, Optimization and NISQ	34
8.1	Intuition: NISQ and Beyond	34
8.2	Quantum Machine Learning	34
8.3	Quantum Chemistry and VQE	35
8.4	Optimization: QAOA	35
8.5	NISQ Limitations and Road Ahead	35
8.6	Python: VQE and QAOA Sketches	36

8.7 Exercises 37

Chapter 1

The Qubit: Beyond the Bit

From 0-or-1 to $\alpha|0\rangle + \beta|1\rangle$ — why nature's bit is a sphere, not a switch.

1.1 Intuition: Why a Bit Is Not Enough

A classical bit is a switch: OFF is 0, ON is 1. Every computer you have used stores essays, photos and videos as strings of such switches. This picture is powerful but incomplete — it describes what we *engineered*, not what nature *allows*.

Quantum mechanics says a particle (an electron spin, a photon polarization, a superconducting loop) can be in a *superposition*: partly 0 and partly 1 at once, with weights that are complex numbers. Until you look, the system *is* both. When you do look, nature rolls dice: you see 0 with some probability, 1 otherwise, and the superposition is gone. This randomness is not ignorance — it is how the world works at small scales.

The qubit is the mathematical model of this idea. It keeps the case “0 or 1” but upgrades the state space from two points to a whole continuum.

1.2 Formalism: The Qubit

We work in $\mathbb{C}^2$ with computational basis

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}. \quad (1.1)$$

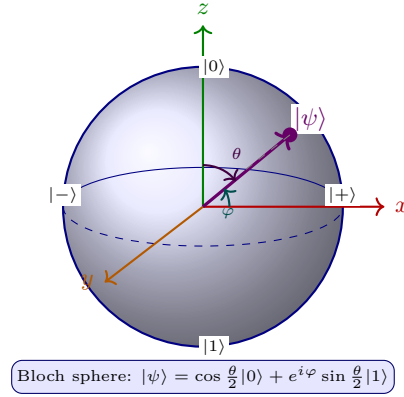
Definition 1.1 (Qubit). A (pure) qubit state is a unit vector

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1. \quad (1.2)$$

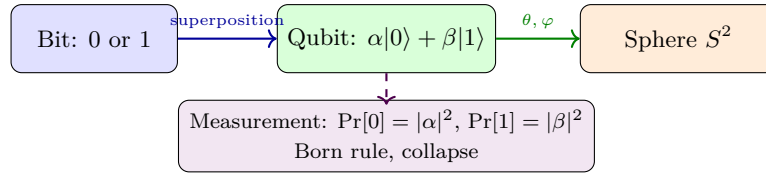
$|\alpha|^2$ is the probability of measuring 0, $|\beta|^2$ of measuring 1. Global phase is unphysical: $e^{i\gamma}|\psi\rangle \equiv |\psi\rangle$.

Writing $\alpha = \cos \frac{\theta}{2}$ and $\beta = e^{i\varphi} \sin \frac{\theta}{2}$ with $\theta \in [0, \pi]$, $\varphi \in [0, 2\pi)$ gives the Bloch representation. Every pure state is a point on the unit sphere.

Definition 1.2 (Bloch sphere). The map $|\psi\rangle \mapsto \mathbf{r} = (\sin \theta \cos \varphi, \sin \theta \sin \varphi, \cos \theta) \in S^2$ is a bijection between pure qubit states modulo phase and points on the Bloch sphere. North pole = $|0\rangle$, south pole = $|1\rangle$, equator = equal superposition.



Measurement in the computational basis projects onto $|0\rangle$ or $|1\rangle$ (Born rule). Measuring a different basis corresponds to rotating the sphere first. Concretely, measuring $|+\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}}$ vs. $|-\rangle = \frac{|0\rangle-|1\rangle}{\sqrt{2}}$ is an X -basis measurement.



1.3 Measurement and Collapse

Theorem 1.1 (Born rule + collapse). Measuring $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ in basis $\{|0\rangle, |1\rangle\}$ yields outcome 0 with probability $|\alpha|^2$ leaving state $|0\rangle$, and outcome 1 with probability $|\beta|^2$ leaving $|1\rangle$.

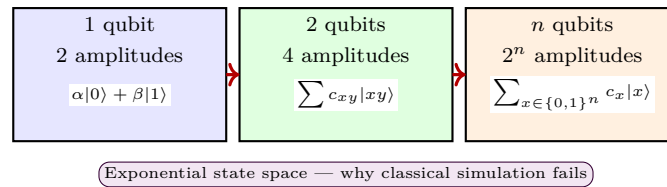
Repeated preparation and measurement estimates $|\alpha|, |\beta|$ but single-shot measurement cannot learn both amplitudes — a *no-cloning* hint. Relative phase φ is invisible in Z -basis statistics but appears in X -basis:

$$\Pr[+] = |\langle +|\psi\rangle|^2 = \frac{1}{2} + \operatorname{Re}(\alpha^* \beta), \quad \Pr[-] = \frac{1}{2} - \operatorname{Re}(\alpha^* \beta). \quad (1.3)$$

Example 1.1 (Interference). $|\psi_1\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}}$ and $|\psi_2\rangle = \frac{|0\rangle-|1\rangle}{\sqrt{2}}$ give identical Z statistics (50/50) but opposite X statistics. Phase matters — it is how quantum algorithms create constructive and destructive interference.

1.4 Multi-Qubit Preview

Two qubits live in $\mathbb{C}^4$ with basis $|00\rangle, |01\rangle, |10\rangle, |11\rangle$. A general state $\sum_{x \in \{0,1\}^2} c_x |x\rangle$ has 4 amplitudes; n qubits have 2^n . This exponential Hilbert space is the source of quantum power — and of simulation hardness.



1.5 Python: Qubits with NumPy

```

1  import numpy as np
2
3  # |0>, |1>, |+>, |-> as vectors
4  ket0 = np.array([1,0], dtype=complex)
5  ket1 = np.array([0,1], dtype=complex)
6  ket_plus = (ket0 + ket1)/np.sqrt(2)
7  ket_minus = (ket0 - ket1)/np.sqrt(2)
8
9  def qubit(alpha, beta):
10     """Create normalized qubit a|0>+b|1>."""
11     v = np.array([alpha, beta], dtype=complex)
12     return v / np.linalg.norm(v)
13
14  def bloch_coords(state):
15     """Return (x,y,z) Bloch vector for pure state."""
16     a, b = state
17     x = 2*np.real(np.conj(a)*b)
18     y = 2*np.imag(np.conj(a)*b)
19     z = np.abs(a)**2 - np.abs(b)**2
20     return (x, y, z)
21
22  def measure_probs(state):
23     """Born rule probabilities in Z basis."""
24     return np.abs(state)**2 # [P0, P1]
25
26  # Example: Hadamard state and measurement
27  psi = qubit(1, 1) # |+>
28  print("psi =", psi)
29  print("Bloch:", bloch_coords(psi)) # ~(1,0,0)
30  print("P0,P1 :", measure_probs(psi)) # [0.5, 0.5]
31  # X-basis probability: |<+|psi>|^2
32  p_plus = abs(np.vdot(ket_plus, psi))**2
33  print("P(+) in X-basis:", p_plus)

```

```

1  # Simulate measurement collapse (Monte Carlo)
2  import random
3
4  def measure(state, shots=1000):
5      p0 = abs(state[0])**2
6      counts = {"0": 0, "1": 0}
7      for _ in range(shots):
8          counts["0" if random.random() < p0 else "1"] += 1
9      return counts

```



```

10
11 psi = qubit(np.cos(0.6), np.exp(1j*0.7)*np.sin(0.6))
12 print("psi Bloch:", bloch_coords(psi))
13 print(measure(psi, 2000))
14
15 # Verify normalization and global phase invariance
16 phase = np.exp(1j*1.23)
17 psi2 = phase * psi
18 print("Same Bloch after phase?",
19       np.allclose(bloch_coords(psi), bloch_coords(psi2)))

```

1.6 Density Matrices and Global Phase

For completeness, a mixed state $\rho = \sum_i p_i |\psi_i\rangle\langle\psi_i|$ has $\text{Tr } \rho = 1$, $\rho \succeq 0$, $\text{Tr } \rho^2 \leq 1$ with equality iff pure. Bloch form for any single-qubit state is

$$\rho = \frac{1}{2}(I + xX + yY + zZ), \quad x^2 + y^2 + z^2 \leq 1, \quad (1.4)$$

so interior of sphere = mixed states. Pure states are surface points. Global phase $e^{i\gamma}$ cancels in $\rho = |\psi\rangle\langle\psi|$, explaining its unphysical nature. The fidelity between ρ and σ is $F = (\text{Tr} \sqrt{\sqrt{\rho}\sigma\sqrt{\rho}})^2$, reducing to $|\langle\psi|\phi\rangle|^2$ for pure states.

Remark 1.1. Relative phase is physical because it changes ρ . Example: $|+\rangle\langle+| \neq |-\rangle\langle-|$ though both are 50/50 in Z basis. This is why interferometers sense phase.

1.7 Exercises

Exercise 1.1. Normalize $|\phi\rangle = (1+i)|0\rangle + (2-i)|1\rangle$ and find $\text{Pr}[0]$, $\text{Pr}[1]$ and its Bloch coordinates (θ, φ) .

Exercise 1.2. Show that $|+\rangle$ and $|-\rangle$ are orthogonal and each lies on the Bloch equator. Which (θ, φ) correspond to $|+i\rangle = \frac{|0\rangle + i|1\rangle}{\sqrt{2}}$?

Exercise 1.3. Prove global phase is unphysical: for any measurement basis $\{|m\rangle\}$, $|\langle m|\psi\rangle|^2 = |\langle m|e^{i\gamma}\psi\rangle|^2$. Find a relative phase that *does* change X -basis statistics.

Exercise 1.4. A qubit is measured in Z basis with outcome 0 with probability $3/4$. Give all possible states (up to global phase) and their Bloch z -coordinates.

Exercise 1.5. Write a Python function that, given n , builds the 2^n -dimensional state vector for $|+\rangle^{\otimes n}$ and verifies that every amplitude equals $2^{-n/2}$. Test for $n = 3$.

Chapter 2

Quantum Circuits: Gates and Entanglement

Classical gates copy and erase. Quantum gates rotate — reversibly, unitarily, and together they entangle.

2.1 Intuition: Reversible Rotations

A classical AND gate takes two bits to one and erases information. Landauer showed erasure costs energy. Quantum evolution is different: a closed quantum system evolves by a *unitary* — a rotation/reflection in Hilbert space that is reversible and length-preserving. No information is lost; computation is a choreography of rotations.

Single-qubit gates rotate the Bloch sphere. Two-qubit gates can do more: they create *entanglement*, correlations stronger than any classical shared randomness. Entanglement is the fuel of quantum speedups.

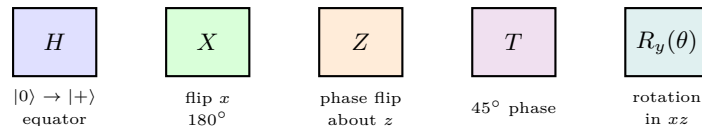
2.2 Single-Qubit Gates

Unitary U satisfies $U^\dagger U = I$. Key gates:

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad (2.1)$$

$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}, \quad T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}, \quad R_\theta = \begin{pmatrix} \cos \frac{\theta}{2} & -\sin \frac{\theta}{2} \\ \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix}. \quad (2.2)$$

X is quantum NOT ($|0\rangle \leftrightarrow |1\rangle$), Z adds phase to $|1\rangle$, H creates superposition: $H|0\rangle = |+\rangle$, $H|1\rangle = |-\rangle$.



All unitary: $U^\dagger U = I$, reversible, $\|U\psi\| = \|\psi\|$

Pauli matrices $\{I, X, Y, Z\}$ form a basis for 2×2 operators. $Y = iXZ$.

2.3 Two-Qubit Gates and Entanglement

Definition 2.1 (CNOT). $\text{CNOT}|c\rangle|t\rangle = |c\rangle|t \oplus c\rangle$. Matrix in basis $|00\rangle, |01\rangle, |10\rangle, |11\rangle$:

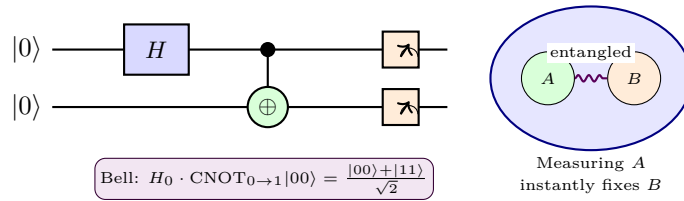
$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}. \quad (2.3)$$

Controlled- U applies U to target iff control is 1. CNOT plus single-qubit gates is universal for unitary circuits.

Definition 2.2 (Entanglement). $|\psi\rangle \in \mathbb{C}^2 \otimes \mathbb{C}^2$ is entangled if it cannot be written as $|a\rangle \otimes |b\rangle$. Bell states are maximally entangled:

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}, \quad |\Phi^-\rangle = \frac{|00\rangle - |11\rangle}{\sqrt{2}}, \quad |\Psi^\pm\rangle = \frac{|01\rangle \pm |10\rangle}{\sqrt{2}}. \quad (2.4)$$

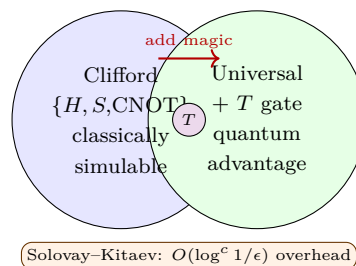
Bell circuit: H on qubit 0 then $\text{CNOT}_{0 \rightarrow 1}$ creates $|\Phi^+\rangle$ from $|00\rangle$.



2.4 Universality

Theorem 2.1 (Universality). $\{H, T, \text{CNOT}\}$ is universal: any n -qubit unitary can be approximated to ϵ with $\text{poly}(n, \log 1/\epsilon)$ gates (Solovay–Kitaev). $\{H, S, \text{CNOT}\}$ generates the Clifford group; adding T gives quantum universality.

Clifford circuits are classically simulable (Gottesman–Knill) — the T gate injects “magic” needed for advantage.



2.5 Python: Gates and Bell States

```

1  import numpy as np
2
3  H = np.array([[1,1],[1,-1]])/np.sqrt(2)
4  X = np.array([[0,1],[1,0]], dtype=complex)
5  Z = np.array([[1,0],[0,-1]], dtype=complex)
6  CNOT = np.array([[1,0,0,0],[0,1,0,0],[0,0,0,1],[0,0,1,0]])
7
8  ket00 = np.array([1,0,0,0], dtype=complex)
9
10 def kron(*args):
11     r = np.array([[1]], dtype=complex)
12     for a in args:
13         r = np.kron(r, a)
14     return r
15
16 # Bell state: (H tensor I) then CNOT
17 HI = kron(H, np.eye(2))
18 phi_plus = CNOT @ HI @ ket00
19 print("Phi+ =", phi_plus) # [0.707,0,0,0.707]
20 # Verify entanglement: reduced density matrix is maximally mixed
21 def partial_trace_first(rho):
22     # trace out second qubit for 2-qubit rho
23     rho = rho.reshape(2,2,2,2)
24     return np.trace(rho, axis1=1, axis2=3)
25
26 rho = np.outer(phi_plus, phi_plus.conj())
27 rhoA = partial_trace_first(rho)
28 print("rho_A =\n", rhoA) # [[0.5,0],[0,0.5]]
29 print("Entangled?", not np.allclose(rhoA @ rhoA, rhoA))

```

```

1  # Qiskit style circuit (also runs with numpy fallback)
2  try:
3      from qiskit import QuantumCircuit
4      qc = QuantumCircuit(2, 2)
5      qc.h(0)
6      qc.cx(0, 1)
7      qc.measure([0,1],[0,1])
8      print(qc.draw(output='text'))
9  except ImportError:
10     print("Qiskit not installed; numpy demo above suffices.")
11     # Numpy measurement simulation
12     probs = np.abs(phi_plus)**2
13     print("Probs |00>,|01>,|10>,|11>:", probs)
14     # Sample
15     outcomes = np.random.choice(4, size=8, p=probs)
16     print("Samples:", [format(o, '02b') for o in outcomes])

```

2.6 Measurement, No-Cloning and Teleportation Preview

No-cloning: no unitary maps $|\psi\rangle|0\rangle \rightarrow |\psi\rangle|\psi\rangle$ for all $|\psi\rangle$ (proof by linearity and inner-product preservation). Teleportation circumvents this by consuming entanglement: shared $|\Phi^+\rangle$ plus two classical bits transmit an unknown qubit without copying.

Theorem 2.2 (No-cloning). No quantum circuit copies an arbitrary unknown state. Formally, no U satisfies $U(|\psi\rangle \otimes |0\rangle) = |\psi\rangle \otimes |\psi\rangle$ for all $|\psi\rangle$.

Measurement in the Bell basis enables teleportation in three steps: Alice Bell-measures her qubit and half of $|\Phi^+\rangle$, sends two bits, Bob applies I, X, Z, XZ accordingly. Entanglement plus classical communication equals quantum communication.

Remark 2.1. Teleportation plus superdense coding (one qubit carries two classical bits using $|\Phi^+\rangle$) show entanglement is a resource interconvertible with communication, formalized by resource theories.

2.7 Exercises

Exercise 2.1. Show $HZH = X$ and $HXH = Z$ by matrix multiplication. Interpret on the Bloch sphere.

Exercise 2.2. Prove $|\Phi^+\rangle$ is entangled: assume $|\Phi^+\rangle = (a|0\rangle + b|1\rangle) \otimes (c|0\rangle + d|1\rangle)$ and derive a contradiction.

Exercise 2.3. Construct a circuit for $|\Psi^-\rangle = \frac{|01\rangle - |10\rangle}{\sqrt{2}}$ from $|00\rangle$ using X, H, CNOT and Z . How many CNOTs are needed?

Exercise 2.4. Show CNOT is its own inverse and is unitary. What is the action of CNOT when the target is $|-\rangle$? (Hint: phase kickback.)

Exercise 2.5. Write Python to verify that the 2-qubit QFT matrix is unitary and that $H \otimes H$ maps $|00\rangle$ to uniform superposition over 4 basis states.

Chapter 3

Early Quantum Advantage: Deutsch–Jozsa and Bernstein–Vazirani

One query to rule them all — when quantum parallelism beats any classical deterministic strategy.

3.1 Intuition: The Power of One Query

Suppose a black box computes $f : \{0, 1\}^n \rightarrow \{0, 1\}$. You may query it. Classically, to decide whether f is constant (all 0s or all 1s) versus balanced (half 0, half 1), you may need $2^{n-1} + 1$ queries in the worst case. Quantumly, one query suffices. No clever classical trick can close this gap — it is information-theoretic.

The trick is *phase kickback* and *interference*. Query f in superposition, the answer is written into phases not bits, then Hadamards interfere amplitudes so that global properties of f become measurable. Deutsch’s problem ($n = 1$) already separates quantum from classical; Deutsch–Jozsa and Bernstein–Vazirani generalize it to n bits.

3.2 Phase Kickback and the Oracle

Classical oracle O_f must be made reversible:

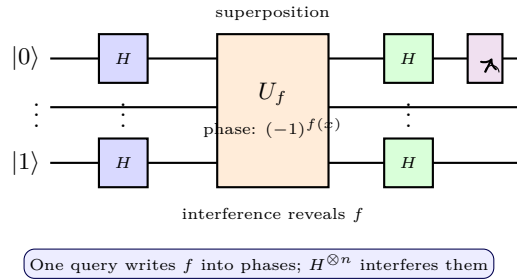
$$U_f|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle, \quad x \in \{0, 1\}^n, y \in \{0, 1\}. \quad (3.1)$$

If the target is $|-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$ then

$$U_f|x\rangle|-\rangle = (-1)^{f(x)}|x\rangle|-\rangle, \quad (3.2)$$

so $f(x)$ appears as a phase. The n -qubit Hadamard $H^{\otimes n}$ creates uniform superposition:

$$H^{\otimes n}|0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0, 1\}^n} |x\rangle. \quad (3.3)$$



3.3 Deutsch and Deutsch–Jozsa

Theorem 3.1 (Deutsch 1985, $n = 1$). With one U_f query, determine $f(0) \oplus f(1)$ (i.e., constant vs. balanced) with certainty. Classically 2 queries are needed deterministically.

Circuit: $|0\rangle|1\rangle \rightarrow H^{\otimes 2} \rightarrow U_f \rightarrow H$ on first qubit $\rightarrow$ measure. Outcome 0 means constant, 1 means balanced.

Theorem 3.2 (Deutsch–Jozsa 1992). Given promise f is constant or balanced on n bits, one quantum query decides which, with certainty. Any deterministic classical algorithm needs $2^{n-1} + 1$ queries. Bounded-error classical needs $O(1)$ but quantum is still exact vs. probabilistic.

After $H^{\otimes n}U_fH^{\otimes n}$, amplitude of $|0\rangle^{\otimes n}$ is $\frac{1}{2^n} \sum_x (-1)^{f(x)}$, which is ± 1 if constant, 0 if balanced.

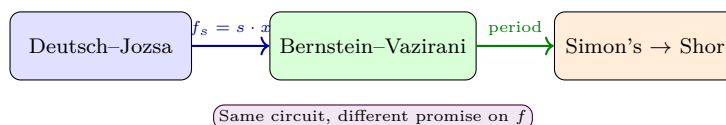
Constant $f \equiv 0$ or 1 $\sum_x (-1)^{f(x)} = \pm 2^n$ $\text{Pr}[0^n] = 1$	Balanced half 0, half 1 $\sum_x (-1)^{f(x)} = 0$ $\text{Pr}[0^n] = 0$	Query count Classical det: $2^{n-1} + 1$ Quantum: 1 (Exponential gap)
---	--	--

3.4 Bernstein–Vazirani

Let $f_s(x) = s \cdot x = \bigoplus_i s_i x_i$ (parity inner product) for hidden $s \in \{0, 1\}^n$.

Theorem 3.3 (Bernstein–Vazirani 1993). One quantum query recovers s exactly. Classically n queries are necessary. The circuit is identical to Deutsch–Jozsa; measurement yields s directly.

After kickback, state is $\frac{1}{\sqrt{2^n}} \sum_x (-1)^{s \cdot x} |x\rangle$, and $H^{\otimes n}$ maps it to $|s\rangle$.



These algorithms are oracular separations, not yet practical speedups, but they isolate the ingredients — superposition, kickback, interference — reused in Grover and Shor.

3.5 Python: Simulating DJ and BV

```

1  import numpy as np
2
3  def hadamard_n(n):
4      H = np.array([[1,1],[1,-1]])/np.sqrt(2)
5      Hn = np.array([[1]], dtype=complex)
6      for _ in range(n):
7          Hn = np.kron(Hn, H)
8      return Hn
9
10 def deutsch_jozsa(f, n):
11     """Simulate DJ for f: {0..2^n-1} -> {0,1}. Return prob of |0^n>."""
12     N = 2**n
13     Hn = hadamard_n(n)
14     # Phase oracle diag((-1)^f(x))
15     phases = np.array([(-1)**f(x) for x in range(N)], dtype=complex)
16     D = np.diag(phases)
17     state0 = np.zeros(N, dtype=complex); state0[0]=1
18     psi = Hn @ D @ Hn @ state0
19     return abs(psi[0])**2 # 1 if constant, 0 if balanced
20
21 # Examples n=3
22 f_const = lambda x: 0
23 f_bal = lambda x: bin(x).count('1') % 2 # parity is balanced
24 print("const P(000):", deutsch_jozsa(f_const, 3)) # ~1
25 print("balanced P(000):", deutsch_jozsa(f_bal, 3)) # ~0

```

```

1  # Bernstein-Vazirani: recover hidden s
2  def bv_circuit(s_int, n):
3      N = 2**n
4      Hn = hadamard_n(n)
5      # f_s(x) = s.x (dot mod2)
6      def f_s(x):
7          return bin(s_int & x).count('1') % 2
8      phases = np.array([(-1)**f_s(x) for x in range(N)], dtype=complex)
9      D = np.diag(phases)
10     state0 = np.zeros(N, dtype=complex); state0[0]=1
11     psi = Hn @ D @ Hn @ state0
12     # measurement: find index with prob 1
13     probs = np.abs(psi)**2
14     s_recovered = int(np.argmax(probs))
15     return s_recovered, probs
16
17 for s in [0b101, 0b011, 0b110]:
18     rec, _ = bv_circuit(s, 3)
19     print(f"hidden {s:03b} -> recovered {rec:03b} match={rec==s}")

```


3.6 Why Oracular Separations Matter

DJ and BV do not give useful speedups on their own — the oracles are artificial. Their value is conceptual: they prove quantum queries carry more information per call than classical bits, and they exhibit the full pipeline reused later:

1. Prepare uniform superposition $H^{\otimes n}|0\rangle$.
2. Encode f into phases via U_f with target $|-\rangle$.
3. Interfere with $H^{\otimes n}$ to map global structure to a computational basis state.
4. Measure to read the answer with certainty.

Grover will replace step 3 with iterative amplitude amplification; Shor will replace it with the quantum Fourier transform. The lower bounds also matter: DJ needs $2^{n-1} + 1$ classical deterministic queries but only 1 quantum query — an exponential gap in exact query complexity. BV shows hidden linear structure s is learnable in one quantum shot versus n classical bits, foreshadowing learning-theoretic advantages in quantum ML.

Example 3.1 (Classical simulation cost). Simulating $H^{\otimes n}$ on $n = 30$ already needs $2^{30} \approx 10^9$ amplitudes — about 16 GB of complex doubles. The textbook’s numpy simulator handles $n \leq 10$ comfortably; beyond that you need a real quantum device or a tensor-network approximation. This exponential overhead is exactly what quantum hardware avoids by storing the state physically.

3.7 Simon and Period Structure

Simon’s problem generalizes parity: $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$ with promise $f(x) = f(y)$ iff $y = x \oplus s$. Quantumly $O(n)$ queries find s ; classically $\Omega(2^{n/2})$ are needed (birthday bound). Simon is the template for period-finding that Shor exploits for $f(x) = a^x \bmod N$ over $\mathbb{Z}$, using QFT instead of Hadamards to handle periods not dividing 2^n . The chain Deutsch $\rightarrow$ Deutsch–Jozsa $\rightarrow$ Bernstein–Vazirani $\rightarrow$ Simon $\rightarrow$ Shor traces increasingly rich hidden structure revealed by Fourier sampling.

Example 3.2 (Simon $n = 2$). If $s = 11$, f might map $00, 11 \rightarrow 0$, $01, 10 \rightarrow 1$. Measuring $A|0\rangle \rightarrow U_f \rightarrow H^{\otimes 2}$ yields random y with $y \cdot s = 0$. Two independent y ’s suffice to solve s by linear algebra over $\mathbb{F}_2$.

3.8 Implementation Cost and Practical Notes

Building U_f depends on f . For DJ/BV the constructions are cheap: BV’s $s \cdot x$ needs n CNOTs, DJ’s arbitrary f may need $O(2^n)$ gates in worst case (encoding a truth table). This is why oracular speedups do not automatically give explicit algorithmic speedups — oracle cost must be counted. In query complexity we count queries; in time complexity we count gates to implement U_f . DJ’s exponential gap in queries collapses if U_f itself is exponentially hard.

Remark 3.1 (Oracle costs). Counting queries hides gate cost. Implementing U_f for arbitrary f may need $O(2^n)$ gates to encode the truth table. Thus an exponential query gap need not imply exponential time gap unless U_f is efficient. For BV, U_{f_s} uses $\text{wt}(s)$ CNOTs, so the gap is both query and time.

Example 3.3 (BV oracle for $s = 101$). For $n = 3$, $s = 101$, $f_s(x) = x_0 \oplus x_2$. Oracle uses $\text{CNOT}_{x_0 \rightarrow y}$ and $\text{CNOT}_{x_2 \rightarrow y}$. With $y = |-\rangle$, phase kickback writes $(-1)^{x_0 \oplus x_2}$. Depth 2, width 4, illustrating linear cost.

Remark 3.2 (Learning perspective). BV shows 1 quantum vs. n classical examples to learn s . This inspired quantum PAC results: discrete-log concepts are quantum-learnable with $O(n)$ examples but classically need $2^{\Omega(n)}$ under assumptions. Same Fourier primitive powers Shor.

3.9 Exercises

Exercise 3.1. Work through Deutsch’s algorithm step-by-step for the four functions $f : \{0, 1\} \rightarrow \{0, 1\}$. Show the final state before measurement is $\pm|0\rangle$ if constant and $\pm|1\rangle$ if balanced.

Exercise 3.2. Prove that any deterministic classical algorithm needs $2^{n-1} + 1$ queries in worst case to solve Deutsch–Jozsa. What about randomized with error $\leq 1/3$? Show $O(1)$ samples suffice with bounded error.

Exercise 3.3. For BV with hidden $s = 1011$, write the state after U_f and after the final $H^{\otimes 4}$. Verify it equals $|s\rangle$. Trace the calculation using $H^{\otimes n}|x\rangle = \frac{1}{\sqrt{2^n}} \sum_y (-1)^{x \cdot y} |y\rangle$.

Exercise 3.4. Implement Simon’s promise ($f(x) = f(y)$ iff $y = x \oplus s$) for $n = 2$ and simulate one quantum query. What linear equations on s do measurements give? How many shots are needed to find $s \neq 0$?

Exercise 3.5. Show BV’s oracle U_{f_s} can be built with at most n CNOTs from the x register to the ancilla, controlled on $s_i = 1$. Draw the circuit for $s = 101$ and $n = 3$.

Chapter 4

Grover's Search: Amplitude Amplification

Searching an unsorted database of size N : classically $N/2$ tries, quantumly $\sqrt{N}$.

4.1 Intuition: Rotating Toward the Needle

You have $N = 2^n$ items, one is marked. An oracle can tell you “yes/no” for any item, but you have no sorted order to exploit. Classically you must try $\Theta(N)$ items. Grover's idea: prepare equal superposition over all N , then repeatedly (a) flip the phase of the marked item and (b) reflect about the average amplitude. Each iteration rotates the state by a fixed angle toward the marked item. After about $\frac{\pi}{4}\sqrt{N}$ rotations you are almost certainly at the answer.

Geometrically the dynamics lives in a 2-D plane spanned by $|w\rangle$ (winner) and $|r\rangle$ (rest). Grover is a rotation in that plane.

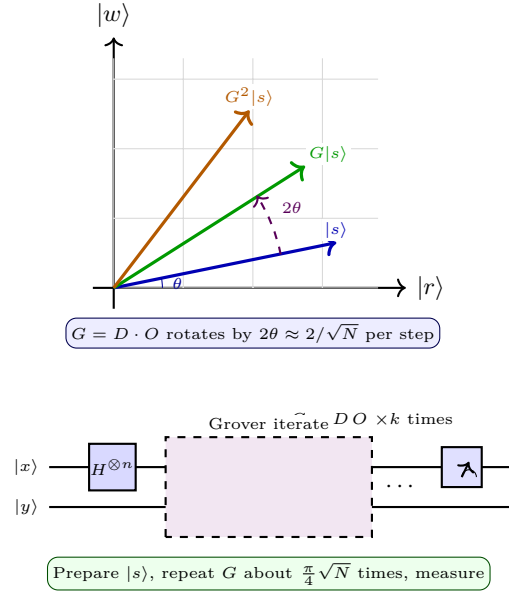
4.2 The Oracle and Diffusion

Let $M = 1$ marked item w , $N = 2^n$. Define

$$|w\rangle \quad \text{and} \quad |r\rangle = \frac{1}{\sqrt{N-1}} \sum_{x \neq w} |x\rangle, \quad |s\rangle = H^{\otimes n} |0\rangle = \frac{1}{\sqrt{N}} \sum_x |x\rangle. \quad (4.1)$$

Write $|s\rangle = \sin \theta |w\rangle + \cos \theta |r\rangle$ where $\sin \theta = 1/\sqrt{N}$.

Oracle O flips phase of the winner: $O|w\rangle = -|w\rangle$, $O|r\rangle = |r\rangle$. Diffusion $D = 2|s\rangle\langle s| - I$ reflects about $|s\rangle$. Grover iterate $G = D \cdot O$ rotates by 2θ in the $\{|r\rangle, |w\rangle\}$ plane.



4.3 Optimal Number of Iterations

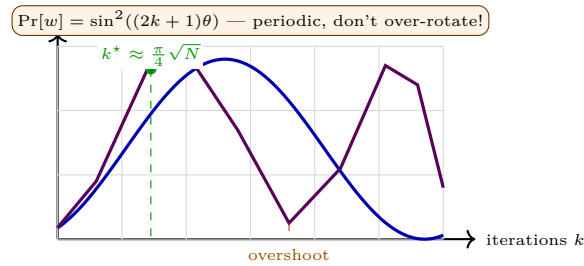
After k iterates,

$$G^k|s\rangle = \sin((2k+1)\theta)|w\rangle + \cos((2k+1)\theta)|r\rangle, \quad (4.2)$$

so success probability is $\sin^2((2k+1)\theta)$.

Theorem 4.1 (Grover optimum). Optimum $k^* \approx \frac{\pi}{4\theta} - \frac{1}{2} \approx \frac{\pi}{4}\sqrt{N}$ gives $\Pr[\text{success}] \approx 1$. Overshooting rotates past $|w\rangle$ and success falls. For M marked items, $\sin \theta = \sqrt{M/N}$ and $k^* \approx \frac{\pi}{4}\sqrt{N/M}$.

Grover is optimal: any quantum algorithm needs $\Omega(\sqrt{N/M})$ queries (BBBV bound). Speedup is quadratic, not exponential, but applies to any unstructured search — 10^6 items: $500\,000 \rightarrow 785$ queries.



4.4 Amplitude Amplification

Grover generalizes: given algorithm A with success probability p , amplification needs $O(1/\sqrt{p})$ repetitions versus $O(1/p)$ classically. Reflection about $A|0\rangle$ replaces Hadamards. This underpins quantum counting, collision finding, and nested Grover for optimization.

4.5 Geometric View and Fixed-Point Variants

Amplitude amplification can be seen as two reflections whose product is a rotation. Reflection O about $|r\rangle$ flips the $|w\rangle$ component; D reflects about $|s\rangle$. Composition of two reflections is a rotation by twice the angle between their axes. This is why Grover is exact analog of classical “inversion about mean” used in some randomized algorithms, but with coherent phases.

Fixed-point search (Yoder–Low–Chuang) avoids overshoot by using varying phases: instead of -1 flips, use $e^{i\phi_j}$ tuned so success probability monotonically increases to $1 - O(\delta)$ without needing to know M exactly. The price is a slightly larger constant factor.

Example 4.1 (When M is unknown). If M is unknown, run exponential probing: try $k = 1, 2, 4, 8, \dots$ Grover steps with random restarts. Expected queries remain $O(\sqrt{N/M})$. Quantum counting (phase estimation on G) estimates M first to $\pm\sqrt{M}$ accuracy using $O(\sqrt{N})$ queries.

4.6 Applications Beyond Search

Grover speeds up any brute-force check: SAT with n variables ($N = 2^n$) from $O(2^n)$ to $O(2^{n/2})$, collision finding $O(N^{1/3})$, and element distinctness $O(N^{2/3})$. In cryptanalysis it halves symmetric key strength: AES-128 offers ~ 64 bits quantum security, hence AES-256 is recommended post-quantum symmetric standard.

4.7 Error Analysis and Reality

Grover assumes perfect oracle and diffusion. With depolarizing error p per gate, fidelity after $k \sim \sqrt{N}$ steps decays as $(1 - p)^{k \cdot G}$ where G gates per iteration. Mitigation via amplitude amplification with error-aware scheduling helps but ultimately fault tolerance is needed for large N . Near-term Grover demonstrations on $n \leq 5$ show the predicted sinusoidal success probability even with NISQ noise, validating the rotation picture.

4.8 Beyond Unstructured Search: Structured Variants

Quantum walk search generalizes Grover to graphs: find a marked vertex with $O(\sqrt{HT})$ where HT is hitting time. For grids and trees this can beat Grover when structure allows faster traversal. The framework unifies Grover, element distinctness, and triangle finding ($O(n^{1.25})$ vs. classical $O(n^2)$).

4.9 Variations and History

Grover (1996) discovered the algorithm by thinking about reflections; Boyer–Brasard–Hoyer–Tapp (1998) analyzed M marks and unknown M . Zalka proved exact optimality. Amplitude amplification (Brassard et al., 2002) generalized to any A with success p . Today Grover is a subroutine in quantum walks, nested search, and QAOA initialization, not a standalone database search, because $O(\sqrt{N})$ oracle calls still require $O(\sqrt{N})$ coherent quantum memory accesses (QRAM) for large N .

4.10 Python: Grover Simulation

```

1 import numpy as np
2
3 def grover_sim(N, w, steps):
4     """Simulate Grover for N items, marked w, for given steps."""
5     n = int(np.log2(N))
6     s = np.ones(N, dtype=complex)/np.sqrt(N) # |s>
7     # Oracles
8     O = np.eye(N, dtype=complex); O[w,w] = -1
9     D = 2*np.outer(s, s.conj()) - np.eye(N)
10    G = D @ O
11    psi = s.copy()
12    for _ in range(steps):
13        psi = G @ psi
14    probs = np.abs(psi)**2
15    return psi, probs
16
17 N, w = 16, 5
18 for k in [0,1,2,3,4]:
19     _, p = grover_sim(N, w, k)
20     print(f"k={k} P(w)={p[w]:.3f} max P={p.max():.3f}")
21
22 # Optimal k ~ pi/4 * sqrt(N)
23 k_opt = int(np.round(np.pi/4*np.sqrt(N)))
24 print("k_opt", k_opt, "P(w) at opt:", grover_sim(N,w,k_opt)[1][w])

```

```

1 # Scaling demo: classical vs quantum queries
2 import math
3 for N in [16, 64, 256, 1024]:
4     classical = N//2
5     quantum = int(math.pi/4*math.sqrt(N))
6     print(f"N={N:4d} classical ~{classical:4d} quantum ~{quantum:3d} speedup {classical
7         /quantum:.1f}x")
8
9 # Multiple marked items
10 N, marked = 16, [3, 7]
11 M = len(marked)
12 theta = math.asin(math.sqrt(M/N))
13 k_opt_M = int(round((math.pi/2 - theta)/(2*theta)))
14 print(f"\nM={M} N={N} theta={theta:.3f} k_opt={k_opt_M}")

```

4.11 Query Lower Bounds

The BBBV theorem shows no quantum algorithm uses $o(\sqrt{N})$ queries for unstructured search. Proof uses hybrid argument: after t queries, amplitudes for oracles differing at one point differ by at most $O(t/\sqrt{N})$; distinguishing needs $t = \Omega(\sqrt{N})$. Thus Grover is asymptotically optimal. For exact success, $\lceil \frac{\pi}{4\theta} - \frac{1}{2} \rceil$ queries are necessary and sufficient. Variants with multiple marked items meet $\Omega(\sqrt{N/M})$ similarly.

Remark 4.1. Grover does not imply $\text{NP} \subseteq \text{BQP}$ with speedup $2^{n/2}$ for k -SAT is still exponential. Quadratic helps

but does not collapse NP. For structured problems, better-than-Grover speedups need structure (e.g., lattice sieving, backtracking with quantum walks).

Remark 4.2 (Optimality sketch). BBBV hybrid argument: let $|\psi_t^f\rangle$ be state after t queries to f . For f, f_w differing at w , $\| |\psi_t^f\rangle - |\psi_t^{f_w}\rangle \| \leq 2t/\sqrt{N}$. Distinguishing needs $\Omega(\sqrt{N})$. Grover is tight; for M marks, $\sqrt{N/M}$.

Example 4.2 (Fixed-point search). Standard Grover overshoots if $k > k^*$. Fixed-point version uses phases ϕ_j in $R_s(\phi) = I - (1 - e^{i\phi})|s\rangle\langle s|$. Chebyshev-chosen ϕ_j makes success monotonic, reaching $1 - \delta$ without knowing M , paying $O(\log 1/\delta)$ overhead.

Remark 4.3 (Quantum counting). Phase estimation on G estimates θ with $\sin \theta = \sqrt{M/N}$, hence M to $O(\sqrt{M})$. Cost $O(\sqrt{N})$. This enables exponential probing when M unknown with expected optimal cost.

4.12 Exercises

Exercise 4.1. For $N = 4$, $w = 3$, simulate Grover by hand: write $|s\rangle$, apply O , then D , and show one iteration finds w with probability 1.

Exercise 4.2. Prove $D = H^{\otimes n}(2|0\rangle\langle 0| - I)H^{\otimes n}$ and explain why reflection about $|0\rangle$ can be built from X gates and a multi-controlled Z .

Exercise 4.3. Derive $k^* \approx \frac{\pi}{4}\sqrt{N/M}$ for M marked items. What is k^* for $N = 10^6$, $M = 10$? What happens if you run twice as many iterations?

Exercise 4.4. Show Grover provides no advantage when $M = N/4$: compute k^* and explain why one iteration already yields $\Pr = 1$ but classical also needs $O(1)$.

Exercise 4.5. Implement amplitude amplification: given unitary A that prepares $\sqrt{p}|good\rangle + \sqrt{1-p}|bad\rangle$, simulate $O(1/\sqrt{p})$ Grover steps and plot success probability vs. steps for $p = 0.05$.

Chapter 5

Shor's Algorithm: Period Finding and Factoring

The algorithm that broke RSA — turning factoring into period finding via the quantum Fourier transform.

5.1 Intuition: Factoring = Period Finding

Factoring $N = pq$ looks unrelated to quantum physics. Shor's insight: it reduces to finding the period r of $f(x) = a^x \bmod N$. If r is even and $a^{r/2} \not\equiv -1 \bmod N$, then $\gcd(a^{r/2} \pm 1, N)$ yields a factor. Classically r can be $\sim N$, requiring exponential time. Quantumly, superposition evaluates f on all x at once and the QFT extracts r by interference.

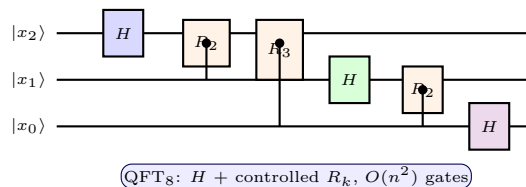
We prepare $\sum_x |x\rangle |f(x)\rangle$; measuring the second register leaves a periodic superposition in the first; QFT turns period in the time domain into peaks in the frequency domain — just as a classical Fourier transform turns a repetitive signal into sharp frequencies.

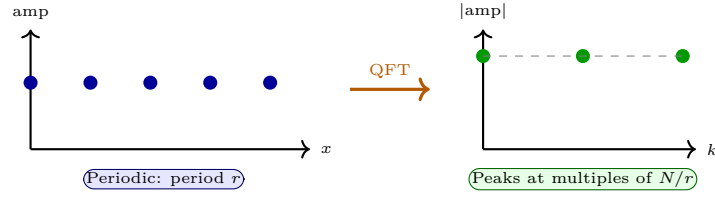
5.2 Quantum Fourier Transform

For $N = 2^n$, the QFT acts as

$$\text{QFT}_N |j\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi i j k / N} |k\rangle, \quad (\text{QFT}_N)_{jk} = \frac{\omega_N^{jk}}{\sqrt{N}}, \quad \omega_N = e^{2\pi i / N}. \quad (5.1)$$

For $n = 1$, QFT is H . For general n , circuit uses H and controlled $R_k = \text{diag}(1, 1, 1, e^{2\pi i / 2^k})$ gates in $O(n^2)$ steps (versus $O(N \log N)$ classical FFT on a vector of size N).

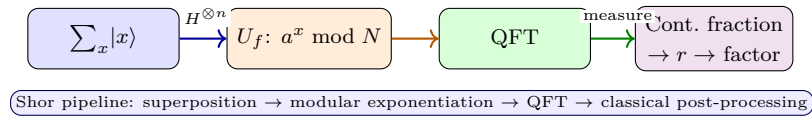




5.3 Shor's Period-Finding Circuit

1. Prepare $\frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} |x\rangle|0\rangle$ with $Q = 2^n \approx N^2$.
2. Apply $U_f : |x\rangle|0\rangle \mapsto |x\rangle|a^x \bmod N\rangle$.
3. Measure second register (optional) giving periodic state $\frac{1}{\sqrt{m}} \sum_j |x_0 + jr\rangle$.
4. QFT on first register, measure to get $k \approx cQ/r$, then continued fractions recover r .

Theorem 5.1 (Shor 1994). Period finding succeeds with probability $\Omega(1)$ using $O(n^2)$ quantum gates. Classical factoring best known is $\exp(O(n^{1/3}))$. Factoring N reduces to period finding with one extra gcd step, so factoring is in BQP.



5.4 Example: Factoring 15

Pick $a = 7$, $N = 15$. Powers $7^x \bmod 15$: 1, 7, 4, 13, 1, ... period $r = 4$ (even). $a^{r/2} = 7^2 = 49 \equiv 4 \bmod 15$, $\gcd(4 - 1, 15) = 3$, $\gcd(4 + 1, 15) = 5$. A $Q = 16$ QFT on a period-4 signal gives peaks at $k = 0, 4, 8, 12$; measurement yields cQ/r and $r = 4$ via continued fractions on k/Q .

5.5 Continued Fractions and Classical Post-Processing

Measurement yields integer k with $|k/Q - c/r| \leq 1/2Q$ for some c . The fraction k/Q approximates c/r . Expanding k/Q as a continued fraction lists convergents p_j/q_j ; the denominator $q_j \approx r$ when $Q \geq N^2$. Testing $a^{q_j} \equiv 1 \bmod N$ recovers r . If c and r share a factor, the convergent gives a divisor of r ; retry with a new k or take lcm of several candidates.

Example 5.1 (Failure modes and retry). Take $N = 21$, $a = 2$, true $r = 6$. If measurement gives $k = 11$, $Q = 64$, then $k/Q = 0.1719 \approx 1/6 = 0.1667$ and continued fraction $11/64 = [0; 5, 1, 4, 2]$ yields convergents $1/5, 1/6$ — success. If $k = 32$, then $k/Q = 1/2$ gives $r = 2$ (a divisor); $a^2 = 4 \not\equiv 1 \bmod 21$, so double to $r = 6$ after checking multiples.

5.6 Shor vs. Grover: Exponential vs. Quadratic

Shor achieves superpolynomial speedup because factoring has hidden periodic structure that QFT exposes. Grover's problem is structureless, so only quadratic is possible (and optimal). This dichotomy guides algorithm design: look for hidden subgroup, hidden shift, or algebraic structure for exponential advantage; use amplitude amplification for generic search and optimization where only quadratic is expected.

5.7 Resource Estimates

Modular exponentiation dominates: $O(n^3)$ gates naively, $O(n^2 \log n)$ with FFT multiplication. For RSA-2048 ($n = 2048$), Shor needs ~ 4000 logical qubits and $\sim 10^9$ Toffoli gates. With distance $d = 27$ surface code ($p = 10^{-3}$), that's $\sim 10^6$ physical qubits and hours of runtime — beyond NISQ but within fault-tolerant roadmaps at 10^{-4} error rates.

5.8 Historical Impact and Cryptography

Shor's 1994 result triggered post-quantum cryptography. RSA, Diffie–Hellman and ECDSA all reduce to hidden subgroup over abelian groups, thus fall to Shor. NIST's 2024 standards (Kyber for KEM, Dilithium for signatures) are lattice-based, conjectured resistant because the hidden subgroup structure is non-abelian or absent. Migrating the internet to PQC is underway before large fault-tolerant machines arrive.

5.9 Python: QFT and Period Finding

```

1  import numpy as np
2
3  def qft_matrix(N):
4      w = np.exp(2j*np.pi/N)
5      j, k = np.meshgrid(np.arange(N), np.arange(N), indexing='ij')
6      return (w**(j*k))/np.sqrt(N)
7
8  def qft_circuit_unitary(n):
9      return qft_matrix(2**n)
10
11 # Verify unitary for n=3
12 for n in [1,2,3]:
13     U = qft_circuit_unitary(n)
14     err = np.linalg.norm(U @ U.conj().T - np.eye(2**n))
15     print(f"n={n} QFT unitary error {err:.2e}")
16
17 # Period finding demo: N=15, a=7, Q=16
18 N_mod, a, Q = 15, 7, 16
19 f = [pow(a, x, N_mod) for x in range(Q)]
20 print("f(x)=7^x mod15:", f) # period 4
21 # Simulate: periodic state with period 4 -> QFT peaks
22 ps = np.zeros(Q, dtype=complex)

```

```

23 for x in range(0, Q, 4): # period-4 comb
24     ps[x] = 1
25 ps /= np.linalg.norm(ps)
26 amps = qft_matrix(Q) @ ps
27 probs = np.abs(amps)**2
28 print("QFT peaks at k:", np.where(probs > 0.1)[0])

1 # Full Shor factoring for N=15 (classical post-processing)
2 import math, random
3
4 def shor_factor(N, a, Q=64):
5     # Quantum part: estimate period r via QFT peaks (simulated)
6     # Find r = order of a mod N classically for demo
7     r = 1
8     while pow(a, r, N) != 1:
9         r += 1
10        if r > N: return None
11    if r % 2 == 1: return None
12    ar2 = pow(a, r//2, N)
13    if ar2 == N-1: return None
14    f1 = math.gcd(ar2-1, N)
15    f2 = math.gcd(ar2+1, N)
16    return r, f1, f2
17
18 for a in [2,7,8,11]:
19     res = shor_factor(15, a)
20     print(f"a={a} -> {res}")
21
22 # Continued fraction to recover r from k/Q
23 from fractions import Fraction
24 k, Q = 48, 64 # example measurement k=48
25 frac = Fraction(k, Q).limit_denominator(15)
26 print(f"k/Q={k}/{Q} -> {frac} => r={frac.denominator}")

```

5.10 From Period to Factor: Number Theory Step

Given even r with $a^{r/2} \not\equiv -1 \pmod{N}$, write $a^r - 1 = (a^{r/2} - 1)(a^{r/2} + 1) \equiv 0 \pmod{N}$. So N divides the product but neither factor alone, hence each shares a non-trivial gcd with N . The number-theoretic lemma: for random a coprime to $N = pq$ with distinct odd primes, $\Pr[r \text{ even and } a^{r/2} \not\equiv -1] \geq 1/2$. Thus a few random a 's succeed with high probability. The full factoring algorithm is Las Vegas: quantum period finding plus classical gcd, poly time overall.

Remark 5.1. Shor also solves discrete log $g^x = h$ by two-dimensional period finding, breaking Diffie–Hellman and elliptic-curve cryptography. Post-quantum candidates (lattices, codes, isogenies) resist known period-finding attacks because their structure is not abelian hidden subgroup.

Remark 5.2 (Continued fractions). Dirichlet: if $|k/Q - c/r| \leq 1/2Q$ and $Q \geq N^2$, then c/r is a convergent of k/Q . Denominators grow exponentially, so $O(\log Q)$ convergents suffice. Testing each q_j via $a^{q_j} \equiv 1$ finds r or a divisor.

Example 5.2 (Coprime failure). If $\gcd(c, r) = d > 1$, convergent gives r/d . Fresh k yields another divisor; lcm recovers r . Failure per shot $\leq 1/\log r$, so $O(\log N)$ repetitions boost success to $1 - 1/\text{poly}(N)$.

Remark 5.3 (Modular exponentiation). Repeated squaring computes $a^x \bmod N$ with $O(n)$ multiplications, each $O(n^2)$ naively or $O(n \log n)$ with FFT. Quantumly a permutation implemented reversibly with $O(n^3)$ Toffoli gates; windowed arithmetic reduces count below 10^9 for $n = 2048$.

5.11 Exercises

Exercise 5.1. Prove QFT is unitary and that $\text{QFT}_2 = H$. How many R_k gates does QFT_{2^n} use? Show an $O(n \log n)$ approximate QFT drops small-angle R_k with error $\leq \epsilon$.

Exercise 5.2. For $f(x) = 7^x \bmod 15$ with $Q = 16$, write the state before QFT after measuring $f = 4$ in the second register. Compute QFT output probabilities and list possible k values.

Exercise 5.3. Given measured $k = 6$, $Q = 16$, use continued fractions on k/Q to find candidate r . Which convergents give the correct period $r = 4$? What if $k = 12$?

Exercise 5.4. Show if r is odd or $a^{r/2} \equiv -1 \bmod N$, Shor's gcd step fails. Estimate the probability a random a coprime to $N = pq$ yields a useful r (at least $1/2$ for distinct odd primes).

Exercise 5.5. Implement modular exponentiation U_f for $N = 21$, $a = 2$ as a permutation matrix on $n = 6$ qubits and verify its action on $|x\rangle|0\rangle$.

Chapter 6

Noise, Decoherence and Quantum Error Correction

Quantum states are fragile. Protecting them is what makes scalable quantum computing possible.

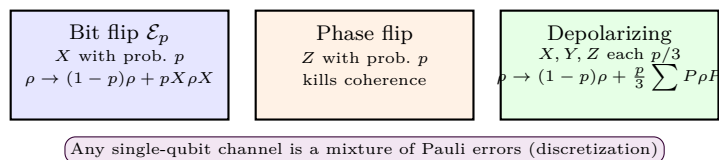
6.1 Intuition: Why Quantum Is Fragile

A classical bit stored as a voltage is robust: small noise does not flip 0 to 1 if thresholds are wide. A qubit stores information in continuous amplitudes α, β and relative phases. Any stray magnetic field, thermal photon, or crosstalk is a tiny unintended rotation. Because amplitudes are continuous, every imperfection matters, and entanglement with the environment looks like *decoherence* — the state becomes mixed and interference dies.

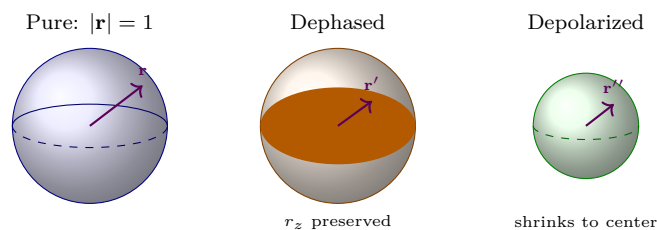
Error correction must fight continuous errors with discrete codes. The surprise: it is possible.

6.2 Decoherence and Noise Models

A pure state $|\psi\rangle$ under noise becomes a density matrix $\rho = \sum_i p_i |\psi_i\rangle\langle\psi_i|$. Common channels:



Bloch sphere picture: bit flip shrinks y - z , phase flip shrinks x - y , depolarizing shrinks uniformly by $1-p$.



T_1 (relaxation) is amplitude damping $|1\rangle \rightarrow |0\rangle$; T_2 (dephasing) is loss of phase with $1/T_2 = 1/2T_1 + 1/T_\phi$.

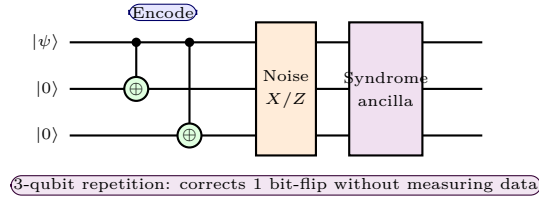
6.3 Three-Qubit Codes

Classical repetition $0 \rightarrow 000$, $1 \rightarrow 111$ corrects one bit flip by majority vote. Quantumly we must not measure the data.

Definition 6.1 (3-qubit bit-flip code). $|0_L\rangle = |000\rangle$, $|1_L\rangle = |111\rangle$. Encoding: $\text{CNOT}_{0 \rightarrow 1}$, $\text{CNOT}_{0 \rightarrow 2}$. Syndrome uses ancilla parity without collapsing $\alpha|0_L\rangle + \beta|1_L\rangle$.

Syndrome table (ancilla $s_1 = Z_0Z_1$, $s_2 = Z_1Z_2$): 00 no error, 10 flip on qubit 0, 11 on qubit 1, 01 on qubit 2. Correction is conditional X . Phase-flip code is the same in H basis: $|\pm_L\rangle = |\pm \pm \pm\rangle$.

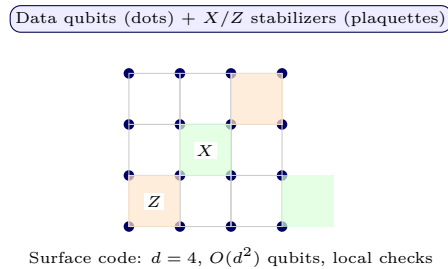
Shor's 9-qubit code concatenates both: corrects any single-qubit error. In general, discretizing errors: correcting X and Z suffices to correct any $aI + bX + cY + dZ$.



6.4 Surface Code and Threshold

Theorem 6.1 (Threshold theorem). If physical error per gate $< p_{\text{th}} \approx 10^{-2}$ (surface code $\sim 1\%$), concatenating codes suppresses logical error $\epsilon_L \sim (p/p_{\text{th}})^{d/2}$ with code distance d , using $\text{polylog}(1/\epsilon)$ overhead. Fault-tolerance is possible.

Surface code arranges qubits on a 2-D lattice; X and Z stabilizers are local plaquette checks. Distance d needs $O(d^2)$ physical qubits per logical qubit. Current devices (50–400 physical qubits, error $\sim 10^{-3}$) are below threshold but not yet at large d .



6.5 Python: Noise and Repetition Code

```

1 import numpy as np
2
3 def bitflip_channel(rho, p):
4     X = np.array([[0,1],[1,0]], dtype=complex)
5     return (1-p)*rho + p*(X @ rho @ X)
6
7 def depolarizing(rho, p):
8     X = np.array([[0,1],[1,0]], complex)
9     Y = np.array([[0,-1j],[1j,0]], complex)
10    Z = np.array([[1,0],[0,-1]], complex)
11    return (1-p)*rho + p/3*(X@rho@X + Y@rho@Y + Z@rho@Z)
12
13 # Bloch shrink demo
14 plus = np.array([[0.5,0.5],[0.5,0.5]], dtype=complex) # |+><+|
15 for p in [0, 0.1, 0.3]:
16     r = depolarizing(plus, p)
17     # Bloch x = Tr(X rho)
18     X = np.array([[0,1],[1,0]], complex)
19     print(f"p={p} <X>={np.trace(X@r).real:.3f} (was 1.0)")
20
21 # 3-qubit bit-flip correction
22 def encode3(psi): # psi=[a,b] -> a|000>+b|111>
23     a,b = psi
24     v = np.zeros(8, dtype=complex); v[0]=a; v[7]=b
25     return v
26
27 # Full 3-qubit syndrome demo
28 import random
29
30 def syndrome_decode(state_vec):
31     """Return syndrome bits for single X error model."""
32     # parity ZOZ1, Z1Z2: measure without collapsing logical
33     probs = np.abs(state_vec)**2
34     # simplified: find which basis state dominates
35     idx = int(np.argmax(probs))
36     bits = format(idx, '03b')
37     # syndrome from bits: s1=b0 xor b1, s2=b1 xor b2
38     b = [int(c) for c in bits]
39     s1, s2 = b[0]^b[1], b[1]^b[2]
40     return (s1, s2), bits
41
42 psi = [1/np.sqrt(2), 1/np.sqrt(2)]
43 code = encode3(psi)
44 print("Encoded |+_L> probs:", np.abs(code)**2)
45 # Simulate X error on qubit 1 (middle)
46 X1 = np.kron(np.kron(np.eye(2), np.array([[0,1],[1,0]])), np.eye(2))
47 err_state = X1 @ code
48 print("After X1 syndrome:", syndrome_decode(err_state))
49 # Correction would apply X1 again
50 recovered = X1 @ err_state
51 print("Recovered fidelity:", abs(np.vdot(code, recovered))**2)

```

6.6 Stabilizer Formalism in Brief

The surface code is a stabilizer code: code space is $+1$ eigenspace of commuting Pauli checks $\{S_i\}$. Measuring S_i gives syndrome without disturbing logical information. Logical operators are strings connecting boundaries, distance is minimal weight of non-trivial logical. Decoding (minimum-weight matching) pairs syndrome defects to infer error. This structure extends to color codes, LDPC codes, and recent good qLDPC advances reducing overhead from $O(d^2)$ toward $O(1)$.

6.7 Exercises

Exercise 6.1. Show bit-flip channel shrinks Bloch vector as $(x, y, z) \rightarrow ((1 - 2p)x, (1 - 2p)y, (1 - 2p)z)$ for depolarizing? Derive the $T_1 \rightarrow \infty$ dephasing map $(x, y, z) \rightarrow ((1 - 2p)x, (1 - 2p)y, z)$.

Exercise 6.2. Encode $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ into 3-qubit code and list the 4 orthogonal subspaces for no error / X_0 / X_1 / X_2 . Why can syndrome be measured without learning α, β ?

Exercise 6.3. Construct the 3-qubit phase-flip code: how does $H^{\otimes 3}$ relate it to the bit-flip code? What syndromes detect Z errors?

Exercise 6.4. For surface code distance $d = 5$ with $p = 10^{-3}$, $p_{\text{th}} = 10^{-2}$, estimate logical error $\sim (p/p_{\text{th}})^{(d+1)/2}$. How large must d be for $\epsilon_L < 10^{-12}$?

Exercise 6.5. Simulate amplitude damping with Kraus operators $E_0 = \text{diag}(1, \sqrt{1 - \gamma})$, $E_1 = \sqrt{\gamma}|0\rangle\langle 1|$. Compute Bloch evolution and show it is not unital.

Chapter 7

Quantum Complexity and Supremacy

Where does quantum sit in the complexity zoo? From P to BQP to QMA, and the quest to prove advantage.

7.1 Intuition: The Complexity Zoo

Classical complexity sorts problems by resources: P (poly time), NP (poly proof verification), PSPACE, etc. Quantum complexity adds BQP (bounded-error quantum poly time) — problems a quantum computer solves efficiently with error $\leq 1/3$ — and QMA (quantum Merlin-Arthur) where the proof itself may be quantum. We know $P \subseteq BQP \subseteq PSPACE$ and $NP \subseteq QMA$, but whether BQP contains NP or vice versa is unknown. The placement matters: if $NP \subseteq BQP$, quantum breaks all NP cryptography; current belief is it does not, but BQP contains factoring and simulation tasks outside P.

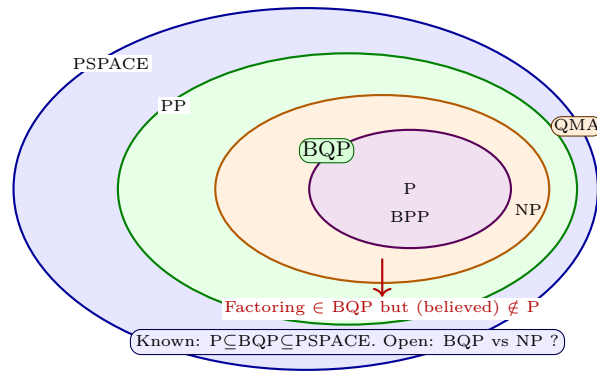
7.2 BQP and Friends

Definition 7.1 (BQP). Language $L \in BQP$ if a uniform poly-size quantum circuit family decides $x \in L$ with $\Pr[\text{correct}] \geq 2/3$ in $\text{poly}(|x|)$ time, with bounded error.

Error $1/3$ is arbitrary: repetition + majority amplifies to $1 - 2^{-\text{poly}}$. BQP is the quantum analogue of BPP (bounded-error classical randomized poly time).

Key inclusions (strictness unknown except $P \neq PSPACE$):

$$P \subseteq BPP \subseteq BQP \subseteq PP \subseteq PSPACE, \quad NP \subseteq QMA, \quad BQP \subseteq QMA. \quad (7.1)$$



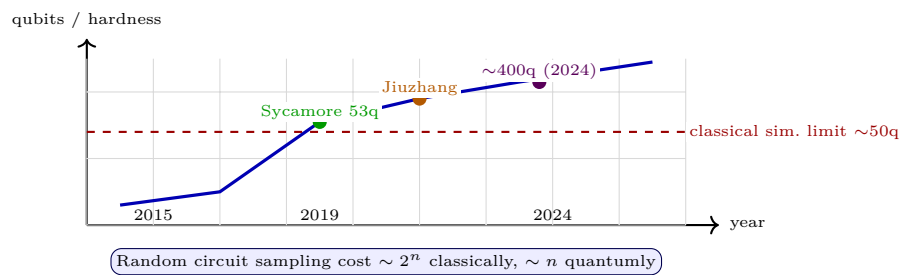
Definition 7.2 (QMA). $L \in QMA$ if a poly-size quantum verifier V exists such that: if $x \in L$, some poly-qubit witness $|\psi\rangle$ makes V accept with $\geq 2/3$; if $x \notin L$, every $|\psi\rangle$ makes V accept with $\leq 1/3$.

QMA-complete problem: local Hamiltonian — estimate ground energy of a k -local quantum system (quantum analogue of SAT). This captures quantum chemistry difficulty.

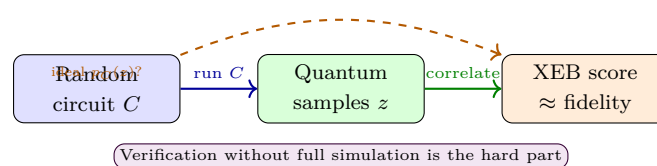
7.3 Quantum Supremacy / Advantage

Supremacy: a quantum device performs a task no classical supercomputer can simulate in feasible time. Advantage: the task is also useful.

Google Sycamore (2019, 53 qubits) sampled random circuits in 200 s; classical estimate 10 000 years (later reduced to days/weeks by better tensor-network simulation, but still exponential). Later photonic boson sampling (Jiuzhang) and IBM’s utility experiments push the frontier.



Verification is the crux: how to check a supremacy experiment without classical simulation? Techniques include cross-entropy benchmarking (XEB), linear XEB, and interactive proofs. Complexity-theoretic evidence: if random circuit sampling were classically easy, the polynomial hierarchy would collapse (Aaronson–Arkhipov, Bremner–Jozsa–Shepherd).



7.4 Oracular Separations

We know relative to an oracle: $\text{BQP} \not\subseteq \text{PH}$ (Raz–Tal 2018, Forrelation problem) and oracles where $\text{BQP} \not\subseteq \text{NP}$ and $\text{NP} \not\subseteq \text{BQP}$. Hence quantum advantage is provable in black-box models, but unconditionally separating BQP from P or NP remains open — as hard as classical $P \neq NP$.

7.5 Beyond BQP: QMA and Local Hamiltonian

QMA captures problems where a quantum witness helps. The k -local Hamiltonian problem: given $H = \sum_i H_i$ with each H_i acting on $\leq k$ qubits and numbers $a < b$ with $b-a \geq 1/\text{poly}(n)$, decide if ground energy $\lambda_{\min} \leq a$ or $\geq b$. Kitaev showed this is QMA-complete for $k \geq 2$, the quantum analogue of Cook–Levin’s SAT completeness for NP. The proof gap $1/\text{poly}$ is needed because quantum measurement is probabilistic.

No good quantum analogue of PH is settled; QPH and BQP/qpoly explore advice. We also know $\text{BQP} \subseteq \text{PP}$: the acceptance probability of a quantum circuit is a PP sum, so quantum does not escape counting complexity.

Example 7.1 (Simulating quantum is hard). Exact simulation of n -qubit circuits costs $O(2^n)$ time/memory. Tensor networks help for shallow or low-entanglement circuits: matrix product states with bond χ cost $O(n\chi^3)$. Random circuits at depth $\sim \sqrt{n}$ maximize entanglement, defeating MPS — hence their use for supremacy.

7.6 Dequantization and Quantum Inspirations

Ewin Tang’s dequantization showed some QML speedups (recommendation systems) were not truly exponential once classical ℓ_2 -sampling assumptions matched quantum state assumptions. This sharpens the search for genuine advantage: problems with structure that resists ℓ_2 dequantization, like quantum simulation and certain kernels, remain promising. Quantum-inspired classical algorithms are a healthy by-product of the quest.

7.7 Interactive Proofs and Delegation

QMA is one round; QIP (quantum interactive proofs) allows interaction and equals PSPACE (like classical $\text{IP}=\text{PSPACE}$). For delegation, Mahadev (2018) gave a protocol for a classical verifier to check a quantum computation assuming LWE hardness, using trapdoor claw-free functions. This bridges complexity theory and near-term verification of supremacy claims without trusting the device.

7.8 Fine-Grained and Average-Case Hardness

Worst-case hardness is not enough for cryptography; average-case matters. For random circuit sampling, Stockmeyer counting arguments plus conjectures about average-case $\#P$ -hardness of approximating output probabilities support hardness. For BQP vs. PH, Raz–Tal’s Forrelation problem gives an oracle separation with only $O(\log n)$ quantum queries vs. $\Omega(\sqrt{n})$ classical — unconditional in query model.

7.9 Open Problems

Is $\text{BPP} \neq \text{BQP}$ unconditionally? Is $\text{NP} \subseteq \text{BQP}$? Does $\text{BQP} \subseteq \text{PH}$? Each would be a major breakthrough. On the practical side, proving a superpolynomial advantage for a *useful* problem (not oracular) with provable classical hardness remains open. Candidates include quantum simulation of strongly correlated materials and certain linear-algebra tasks under plausible assumptions.

7.10 Proof Techniques

Proving BQP lower bounds uses polynomial method (Beals et al.) and adversary method (Ambainis). Upper bounds use circuit constructions. For Forrelation, Raz–Tal combine Fourier analysis with pseudo-randomness to show classical decision trees need $\tilde{\Omega}(\sqrt{N})$ queries while one quantum query suffices, separating BQP from PH relative to an oracle.

Remark 7.1 (Why separations are hard). Unconditional $\text{BQP} \neq \text{BPP}$ would imply $\text{P} \neq \text{PSPACE}$ -level breakthroughs. Current gaps are conditional on assumptions (e.g., factoring not in BPP) or oracular. This mirrors classical $\text{P} \neq \text{NP}$ hardness and drives interest in fine-grained and average-case evidence.

7.11 Python: Complexity Scaling Demo

```

1 import math, random
2
3 def classical_random_circuit_cost(n, depth):
4     """Rough ops to simulate n-qubit random circuit exactly:  $O(2^n)$ ."""
5     return 2**n * depth
6
7 def quantum_random_circuit_cost(n, depth):
8     return n * depth # linear in qubits
9
10 for n in [10, 20, 30, 50, 53]:
11     c = classical_random_circuit_cost(n, 20)
12     q = quantum_random_circuit_cost(n, 20)
13     print(f"n={n:2d}   classical ~2^{n}={c:.1e} ops   quantum ~{q} ops   ratio {c/q:.1e}")
14
15 # Forrelation gap (Raz-Tal): quantum 1 query vs classical ~sqrt(N)
16 N = 1024
17 print(f"\nForrelation N={N}: quantum 1 query, classical ~{int(math.sqrt(N))} queries")

```

```

1 # XEB fidelity estimation sketch
2 import numpy as np
3
4 def xeb_fidelity(samples, ideal_probs):
5     """Linear XEB:  $F \sim N * \text{mean}(p_{\text{ideal}}(\text{samples})) - 1$ ."""
6     N = len(ideal_probs)
7     mean_p = np.mean([ideal_probs[s] for s in samples])
8     return N * mean_p - 1
9
10 # Ideal Porter-Thomas (random circuit) probs: exponential distribution

```

```

11 N = 1024
12 rng = np.random.default_rng(0)
13 ideal = rng.exponential(scale=1/N, size=N); ideal /= ideal.sum()
14 # Perfect quantum sampler
15 samples_perfect = rng.choice(N, size=5000, p=ideal)
16 # Noisy sampler (mix with uniform)
17 noisy_p = 0.7*ideal + 0.3*np.ones(N)/N
18 samples_noisy = rng.choice(N, size=5000, p=noisy_p)
19
20 print(f"XEB perfect ~{xeb_fidelity(samples_perfect, ideal):.3f} (expect 1.0)")
21 print(f"XEB noisy ~{xeb_fidelity(samples_noisy, ideal):.3f} (expect ~0.7)")
22 print(f"Uniform XEB ~{xeb_fidelity(rng.choice(N,5000), ideal):.3f} (expect 0)")

```

7.12 Unconditional vs. Conditional Gaps

We have no unconditional proof that $\text{BPP} \neq \text{BQP}$ or $\text{P} \neq \text{PSPACE}$ implies $\text{BPP} \neq \text{BQP}$ open. Yet oracle separations are unconditional relative to that oracle. NREL? Also, $\text{BQP} \subseteq \text{PP}$ barrier suggests quantum advantage cannot be too strong: PP is in PSPACE , so exponential advantage beyond PP would place BQP outside PP , contradicting inclusion. The frontier is fine-grained: polynomial vs. super-polynomial separations.

Current research focuses on fine-grained advantage: e.g., $O(n^2)$ quantum vs. $\Omega(n^3)$ classical for linear systems (HHL) under sparsity and condition-number assumptions, or sample complexity gaps in learning theory where quantum PAC learning can be exponentially better for certain concept classes built from cryptography.

Remark 7.2 (BQP in PP). Acceptance probability $\Pr[\text{accept}] = \sum_z |\langle z|C|0\rangle|^2$ is a sum of exponentially many P -computable terms, a PP predicate: accept iff $\Pr > 1/2$. Thus $\text{BQP} \subseteq \text{PP} \subseteq \text{PSPACE}$, bounding quantum advantage unconditionally.

Example 7.2 (Tensor-network simulation). Depth d 1-D circuits have entanglement $O(d)$, simulable via MPS with bond $\chi = 2^{O(d)}$ and cost $O(n\chi^3)$. Random circuits at depth $\approx \sqrt{n}$ generate volume law $\chi \sim 2^{n/2}$, defeating MPS. Hence supremacy circuits pick depth ~ 20 for $n = 53$.

Remark 7.3 (Dequantization). Tang's ℓ_2 -sampling algorithm matched quantum $O(\text{poly log } N)$ for recommendation under QRAM-like sampling assumptions. Lesson: exponential gaps need input-model qualification. Genuine advantage problems use structure like periodic or algebraic oracles that resist ℓ_2 shortcuts.

7.13 Exercises

Exercise 7.1. Show error $1/3$ in BQP can be amplified to 2^{-k} by $O(k)$ repetitions and majority vote. Why does this fail if error is $1/2$?

Exercise 7.2. Explain why factoring $\in \text{BQP}$ does not imply $\text{NP} \subseteq \text{BQP}$. What would $\text{NP} \subseteq \text{BQP}$ mean for lattice-based cryptography?

Exercise 7.3. State the local Hamiltonian problem and explain why it is QMA -complete (analogue of Cook–Levin). What parameter controls the promise gap?

Exercise 7.4. A 60-qubit random circuit has ideal output probabilities $\sim$ Porter-Thomas. Estimate memory to store the full 2^{60} state vector in double precision. Why does this motivate XEB?

Exercise 7.5. Research the Sycamore vs. classical simulation debate: how did tensor-network improvements change the 10 000-year claim? What does this teach about supremacy definitions?

Chapter 8

Applications: QML, Chemistry, Optimization and NISQ

From noisy 50-qubit chips to fault-tolerant dreams — where quantum may actually help.

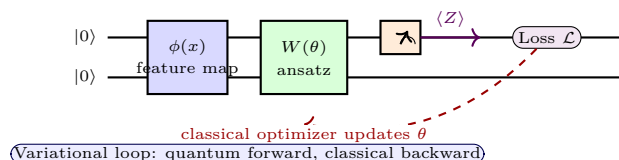
8.1 Intuition: NISQ and Beyond

We have 50–400 qubit devices that are noisy and lack error correction. This is the NISQ era (Noisy Intermediate-Scale Quantum, Preskill 2018). They cannot run Shor on RSA-2048 (needs ~ 4000 logical $\approx$ millions physical qubits). But they can try variational algorithms: a quantum circuit with tunable parameters prepares a state, a classical optimizer adjusts parameters to minimize a cost — hybrid quantum-classical loops that tolerate some noise.

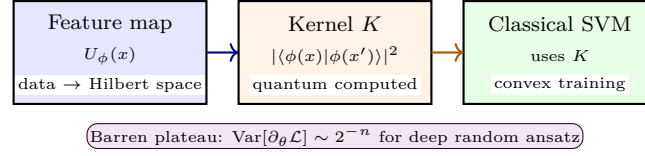
The big application bets are: quantum machine learning (QML), quantum chemistry/materials, and combinatorial optimization.

8.2 Quantum Machine Learning

Data x is encoded into a quantum feature map $|\phi(x)\rangle = U_\phi(x)|0\rangle$. A parameterized circuit $W(\theta)$ processes it; measurement yields prediction. Training minimizes loss over θ .



Quantum kernel methods use $K(x, x') = |\langle \phi(x) | \phi(x') \rangle|^2$ computed on hardware, then classical SVM. Advantage requires feature maps classically hard to simulate and datasets with quantum structure; for generic classical data, barren plateaus (gradients vanish exponentially in n) can kill training.



8.3 Quantum Chemistry and VQE

Molecular ground energy $E_0 = \min_{|\psi\rangle} \langle \psi | H | \psi \rangle$ where H is the electronic Hamiltonian. VQE (variational quantum eigensolver) uses ansatz $|\psi(\theta)\rangle$ and minimizes $\langle H \rangle$ measured termwise.

$$H = \sum_P h_P P, \quad P \in \{I, X, Y, Z\}^{\otimes n}, \quad \langle H \rangle = \sum_P h_P \langle \psi(\theta) | P | \psi(\theta) \rangle. \quad (8.1)$$

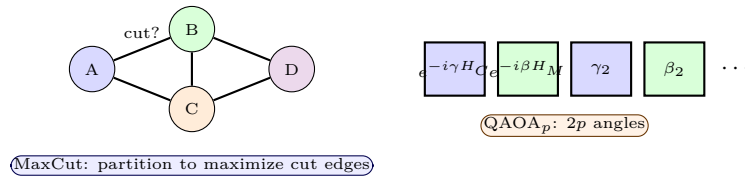
Each $\langle P \rangle$ is estimated by measuring that Pauli string. *FeMoco*, catalysts, and battery materials are target use cases; current VQE handles 10–20 orbitals with noise mitigation.

8.4 Optimization: QAOA

For MaxCut, cost $C(z) = \sum_{(i,j) \in E} \frac{1}{2}(1 - z_i z_j)$, $z_i \in \{-1, 1\}$. Map to Hamiltonian $H_C = \sum_{(i,j)} \frac{1}{2}(1 - Z_i Z_j)$. QAOA alternates

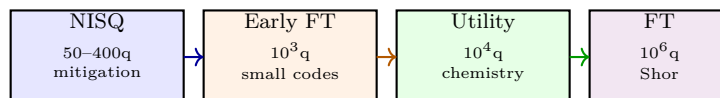
$$|\gamma, \beta\rangle = e^{-i\beta_p H_M} e^{-i\gamma_p H_C} \dots e^{-i\beta_1 H_M} e^{-i\gamma_1 H_C} |+\rangle^{\otimes n}, \quad (8.2)$$

with mixer $H_M = \sum_i X_i$, then optimizes angles γ, β classically. At depth $p \rightarrow \infty$, QAOA contains adiabatic evolution and is universal; at small p it is a NISQ heuristic.



8.5 NISQ Limitations and Road Ahead

NISQ devices lack correction: depth limited to ~ 10 – 100 gates before noise dominates. Error mitigation (zero-noise extrapolation, Clifford data regression) stretches reach but not exponentially. Fault tolerance needs 10^3 – 10^4 physical per logical qubit at 10^{-3} error. The path: better qubits $\rightarrow$ error mitigation $\rightarrow$ early correction $\rightarrow$ utility-scale advantage in chemistry and materials before breaking RSA.



8.6 Python: VQE and QAOA Sketches

```

1  import numpy as np
2  from scipy.optimize import minimize
3
4  # VQE toy:  $H = Z_0 Z_1 + 0.5 X_0 + 0.5 X_1$ , 2 qubits
5  Z = np.array([[1,0],[0,-1]], complex)
6  X = np.array([[0,1],[1,0]], complex)
7  I = np.eye(2)
8  H = np.kron(Z,Z) + 0.5*np.kron(X,I) + 0.5*np.kron(I,X)
9  eigvals = np.linalg.eigvalsh(H)
10 print(f"Exact ground energy {eigvals[0]:.4f}")
11
12 def ansatz(theta):
13     """Simple 2-qubit ansatz: Ry on each + CNOT."""
14     ry = lambda t: np.array([[np.cos(t/2), -np.sin(t/2)],
15                               [np.sin(t/2), np.cos(t/2)]])
16     Ry0 = np.kron(ry(theta[0]), I)
17     Ry1 = np.kron(I, ry(theta[1]))
18     CNOT = np.array([[1,0,0,0],[0,1,0,0],[0,0,0,1],[0,0,1,0]], complex)
19     psi0 = np.array([1,0,0,0], complex)
20     return CNOT @ Ry1 @ Ry0 @ psi0
21
22 def vqe_cost(theta):
23     psi = ansatz(theta)
24     return np.real(psi.conj() @ H @ psi)
25
26 res = minimize(vqe_cost, x0=[0.5, -0.3], method='COBYLA')
27 print(f"VQE energy {res.fun:.4f} at {res.x}")

```

```

1  # QAOA MaxCut p=1 on 4-node cycle
2  import numpy as np
3  from scipy.optimize import minimize
4
5  # 4-cycle edges
6  edges = [(0,1),(1,2),(2,3),(3,0)]
7  n = 4; N = 2**n
8
9  def cost_hamiltonian():
10     Hc = np.zeros((N,N), complex)
11     for i,j in edges:
12         for z in range(N):
13             bits = [(z>>k)&1 for k in range(n)]
14             zi, zj = 1-2*bits[i], 1-2*bits[j] # +1/-1
15             Hc[z,z] += 0.5*(1 - zi*zj)
16     return Hc
17
18 Hc = cost_hamiltonian()
19 # p=1 QAOA expectation
20 def qaoa_expect(params):
21     gamma, beta = params
22     #  $|+\rangle^{\otimes n}$ 
23     plus = np.ones(N, complex)/np.sqrt(N)

```

```

24 # e^{-i gamma Hc} diagonal, e^{-i beta sum X} = product Ry-like
25 diag = np.exp(-1j*gamma*np.diag(Hc))
26 psi = diag * plus
27 # Mixer: apply Rx(2beta) on each qubit via brute force
28 # Simplified: approximate by exponentiating sum X (small n)
29 X = np.array([[0,1],[1,0]], complex)
30 Xm = sum(np.kron(np.kron(np.eye(2**i), X), np.eye(2**(n-1-i)))
31          for i in range(n))
32 Um = np.linalg.matrix_power(np.eye(N) -1j*beta*Xm, 1) # first order
33 # Use exact expm for accuracy
34 from scipy.linalg import expm
35 Um = expm(-1j*beta*Xm)
36 psi = Um @ psi
37 return -np.real(psi.conj() @ Hc @ psi) # negative for minimize
38
39 res = minimize(qaoa_expect, x0=[0.5, 0.5], method='COBYLA')
40 print(f"QAOA p=1 best cut {-res.fun:.3f} (opt=4) at gamma,beta={res.x}")

```

8.7 Exercises

Exercise 8.1. For VQE with $H = Z \otimes Z$, what is the single-qubit R_y ansatz minimum? Show entanglement (CNOT) is not needed for this H but is for $H = X \otimes X + Y \otimes Y$.

Exercise 8.2. MaxCut on a triangle (3 nodes, 3 edges): enumerate all cuts, find optimum, then simulate QAOA $p = 1$ and plot $\langle H_C \rangle$ over $\gamma, \beta \in [0, \pi]$.

Exercise 8.3. Explain barren plateaus: for a random n -qubit ansatz, show $\text{Var}[\partial_k \mathcal{L}]$ decays exponentially in n and discuss how local cost functions and shallow depth mitigate it.

Exercise 8.4. Compare quantum kernel $K_{ij} = |\langle \phi(x_i) | \phi(x_j) \rangle|^2$ with classical RBF. When would you expect quantum advantage? Propose a dataset built from discrete-log structure.

Exercise 8.5. Estimate physical qubits needed for Shor on RSA-2048: assume distance $d = 27$ surface code, $2d^2$ physical per logical, ~ 4000 logical qubits. What error budget does this imply?